ShowUp: A Relational Database–Driven Campus Event Discovery, Registration and Check-in System

Rayna Agarwal
Registration No. 25BCE0703
Department of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Database Management Systems — Course Project Report · Source code: github.com/raynaagarwal01-rgb/ShowUp

***Abstract*—College events such as hackathons, workshops, competitions and cultural programmes are still announced and registered through a patchwork of Google Forms, spreadsheets and messaging groups. This produces duplicated and inconsistent data, over-subscribed events, lost team codes and paper-based attendance. This report presents ShowUp, a web application for discovering events across colleges, registering solo or as a team, receiving a QR ticket and being checked in at the venue. The work follows the complete database development life cycle: requirements were identified and analysed; a conceptual Entity–Relationship (ER) model with an Enhanced ER (EER) specialization was constructed; the model was mapped to a relational schema and verified to be in Third Normal Form (3NF) and Boyce–Codd Normal Form (BCNF); the schema was implemented in MySQL 8.0 as eight InnoDB tables with twelve foreign keys and declarative CHECK, UNIQUE and NOT NULL constraints; and the database was populated with representative data. A React and TypeScript front end communicates with a Node.js/Express application tier that issues only parameterised SQL through a connection pool, and enforces business rules—deadlines, capacity, duplicate prevention and team-size limits—inside ACID transactions that use row-level locks. Thirty-five test cases covering authentication, registration, teams, check-in, database constraints, concurrency and SQL-injection resistance were executed against a live MySQL 8.0.46 server and all passed, including a race of five simultaneous registrations for a single remaining seat.**

***Index Terms*—Database design, entity–relationship model, EER specialization, relational mapping, normalization, MySQL, SQL DDL/DML, transactions, concurrency control, web application, event management.**

I. Introduction

University campuses host a large number of co-curricular events every semester. At institutions such as VIT Vellore a student may, in a single week, be invited to a 24-hour hackathon, a design workshop, an e-sports tournament and a guest lecture, each organised by a different club and each using its own registration form. Organisers, in turn, export responses to spreadsheets, try to reconcile duplicate submissions, chase incomplete teams and mark attendance on paper at the venue.

These are fundamentally *data management* problems: the same student is stored many times in unrelated files, there is no single authoritative count of seats taken, team membership is captured as free text, and nothing prevents two people from taking the last seat at the same moment. A relational database management system (RDBMS) is designed precisely to remove such redundancy and to enforce integrity and consistency under concurrent access [1], [2].

ShowUp addresses the problem with a single web platform backed by a normalized MySQL database. Students browse events from several colleges, register individually or form a team that friends join with a six-character code, and receive a QR ticket. Organisers create and edit events, view participants grouped by team, post announcements, answer questions, publish results and check participants in by scanning their tickets.

A. Objectives

- Identify and document the functional, non-functional and data requirements of campus event registration.
- Design a conceptual ER/EER model and map it systematically to a relational schema.
- Normalize every relation to at least 3NF and verify BCNF using functional dependencies.
- Implement the schema with SQL DDL, populate it with SQL DML and enforce integrity declaratively wherever possible.
- Build a usable front end and an application tier that implements business rules transactionally.
- Validate the system with documented test cases, including constraint and concurrency tests.

B. Scope

The project covers user accounts (students and organisers), events, solo and team registration, QR tickets and check-in, announcements, questions and answers, and results. Online payment, e-mail/SMS notification and waitlists are deliberately out of scope; the event fee is stored for information only. The database and back end were kept intentionally small—eight tables and plain SQL with no Object–Relational Mapper (ORM)—so that every design decision can be traced from requirement to table to query.

C. Organization of the Report

The report follows the eleven stages of the project. Section II analyses the problem and requirements; Section III constructs the ER/EER diagrams; Section IV maps them to the relational model; Section V normalizes the relations; Sections VI and VII present the DDL and DML; Section VIII describes the front end; Section IX the business logic; Section X database connectivity; Section XI testing and validation; and Section XII the demonstration and documentation. Section XIII concludes.

II. Problem Identification and Requirement Analysis

A. Problem Statement

Design and implement a database-driven system that allows students of multiple colleges to discover events and register for

them individually or as teams, and that allows organisers to manage events and attendance, while guaranteeing that (i) no student is registered twice for the same event, (ii) no event exceeds its capacity even under simultaneous registrations, (iii) registrations close at the deadline, (iv) teams respect minimum and maximum sizes, and (v) all data about a person, event or team is stored exactly once.

B. Analysis of the Existing Process

Interviews with club coordinators and inspection of typical registration sheets revealed the following deficiencies, each of which maps to a database concept:

- **Redundancy and update anomalies**—a student's name, e-mail and registration number are retyped on every form; correcting one copy leaves the others stale.
- **No integrity constraints**—forms accept duplicate submissions, invalid team sizes and registrations after the deadline.
- **No concurrency control**—capacity is checked manually after the fact, so popular events are routinely over-subscribed.
- **Repeating groups**—team members are typed into one cell ("Ananya, Rohan, …"), which cannot be queried or validated.
- **No single source of truth for attendance**—check-in is done on paper and never linked back to the registration.

C. Stakeholders

Three actors were identified. A *student* (participant) browses, registers, forms or joins teams, asks questions and presents a ticket. An *organizer* (club or society account) creates and manages its own events and checks participants in. A *visitor* (not signed in) sees the landing page and may sign up. Following IEEE Std 830 practice [6], requirements are numbered so that they can be traced to design elements and test cases.

TABLE I
FUNCTIONAL REQUIREMENTS

ID	Requirement	Actor
FR-1	Sign up and log in with e-mail and password; choose role student or organizer	All
FR-2	List all events with organizer, college and live seats left; search, filter and sort	Student
FR-3	View event detail: description, venue, date, deadline, team size, fee	Student
FR-4	Register for a solo event before the deadline if seats remain	Student
FR-5	Create a team for a team event and receive a unique join code	Student
FR-6	Join a team with its code, subject to the team's maximum size	Student
FR-7	Cancel a registration; leave the team; free the seat	Student
FR-8	View own registrations with QR ticket and team details	Student
FR-9	Create, edit and delete own events	Organizer
FR-10	View participants grouped by team; export CSV	Organizer
FR-11	Check participants in by scanning or typing the ticket number	Organizer
FR-12	Post announcements, answer questions, publish winners	Organizer
FR-13	Ask a question about an event	Student

TABLE II
NON-FUNCTIONAL REQUIREMENTS

ID	Category	Requirement
NFR-1	Integrity	Keys, foreign keys, CHECK and UNIQUE constraints enforced by the DBMS, not only by code
NFR-2	Consistency	Multi-statement operations are atomic (ACID transactions)
NFR-3	Concurrency	Capacity can never be exceeded by simultaneous requests
NFR-4	Security	Passwords stored as bcrypt hashes; all SQL parameterised; role-based authorization
NFR-5	Usability	Responsive UI; actions complete in at most three clicks from the event page
NFR-6	Maintainability	Plain SQL, small route files, schema scripts that are safe to re-run
NFR-7	Portability	Runs on any OS with Node.js 20+ and MySQL 8.0.16+

D. Data Requirements

From the functional requirements the following data must be stored: for every *user*—name, unique e-mail, password hash, role, optional college and registration number; for every *event*—organizer, title, description, category, venue, start date-time, registration deadline, capacity, fee, and minimum/maximum team size; for every *team*—event, name (unique within the event), globally unique join code and leader; *membership* of users in teams; for every *registration*—participant, event, optional team, status and check-in time; and per event, *announcements*, *questions* with optional answers and *winners* with position and prize. Seats left is a derived value and is not stored.

TABLE III
HARDWARE AND SOFTWARE REQUIREMENTS

Layer	Specification
DBMS	MySQL Community Server 8.0 (tested on 8.0.46), InnoDB engine
Server	Node.js 20+ with Express 5, mysql2, bcryptjs, jsonwebtoken
Client	React 19, TypeScript 6, Vite 8, Tailwind CSS 4, qrcode, html5-qrcode
Tools	MySQL Workbench / mysql CLI, VS Code, Git and GitHub
Hardware	Any PC with 4 GB RAM; phone camera for QR check-in (optional)

III. CONSTRUCTION OF ER/EER DIAGRAM

The conceptual model was built with Chen's ER notation [4] and the EER extensions described by Elmasri and Navathe [1]. Nouns in the requirements became candidate entities, verbs became relationships, and descriptive nouns became attributes.

A. Entities and Attributes

TABLE IV
ENTITY SETS AND ATTRIBUTES

Entity	Key	Other attributes
USER	id	name, email (unique), password_hash, role, college, reg_no, created_at
EVENT	id	title, description, category, venue, event_date, reg_deadline, capacity, fee, team_min, team_max, created_at; *seats_left* (derived)
TEAM	id	name, join_code (unique), created_at
REGISTRATION	id	status, checked_in_at, registered_at (associative entity)
ANNOUNCEMENT	id	title, content, created_at
EVENT_QUESTION	id	question, answer (optional), created_at
EVENT_WINNER	id	position, winner_name, prize (optional)

B. Relationships, Cardinality and Participation

TABLE V
RELATIONSHIP SETS

Relationship	Entities	Card.	Participation
ORGANIZES	USER – EVENT	1:N	EVENT total
REGISTRATION	USER – EVENT	M:N	partial both
PART_OF	REGISTRATION – TEAM	N:1	REGISTRATION partial
MEMBER_OF	USER – TEAM	M:N	TEAM total
LEADS	USER – TEAM	1:N	TEAM total
FORMS	EVENT – TEAM	1:N	TEAM total
POSTS	EVENT – ANNOUNCEMENT	1:N	ANNOUNCEMENT total
ASKS	USER – EVENT_QUESTION	1:N	QUESTION total
ABOUT	EVENT – EVENT_QUESTION	1:N	QUESTION total
HAS_RESULT	EVENT – EVENT_WINNER	1:N	WINNER total

REGISTRATION is modelled as an *associative entity* (a relationship promoted to an entity) because it carries its own attributes—status, check-in time and timestamp—and itself participates in the PART_OF relationship with TEAM. Its identifier is printed in the QR ticket (`SHOWUP-REG-<id>`). MEMBER_OF carries the attribute *joined_at*, which is used to choose a new leader when the current leader leaves.

C. ER Diagram

The complete ER diagram is shown in Fig. 2. Double lines denote total participation; for example, every event must be organized by exactly one user and every team must belong to one event.

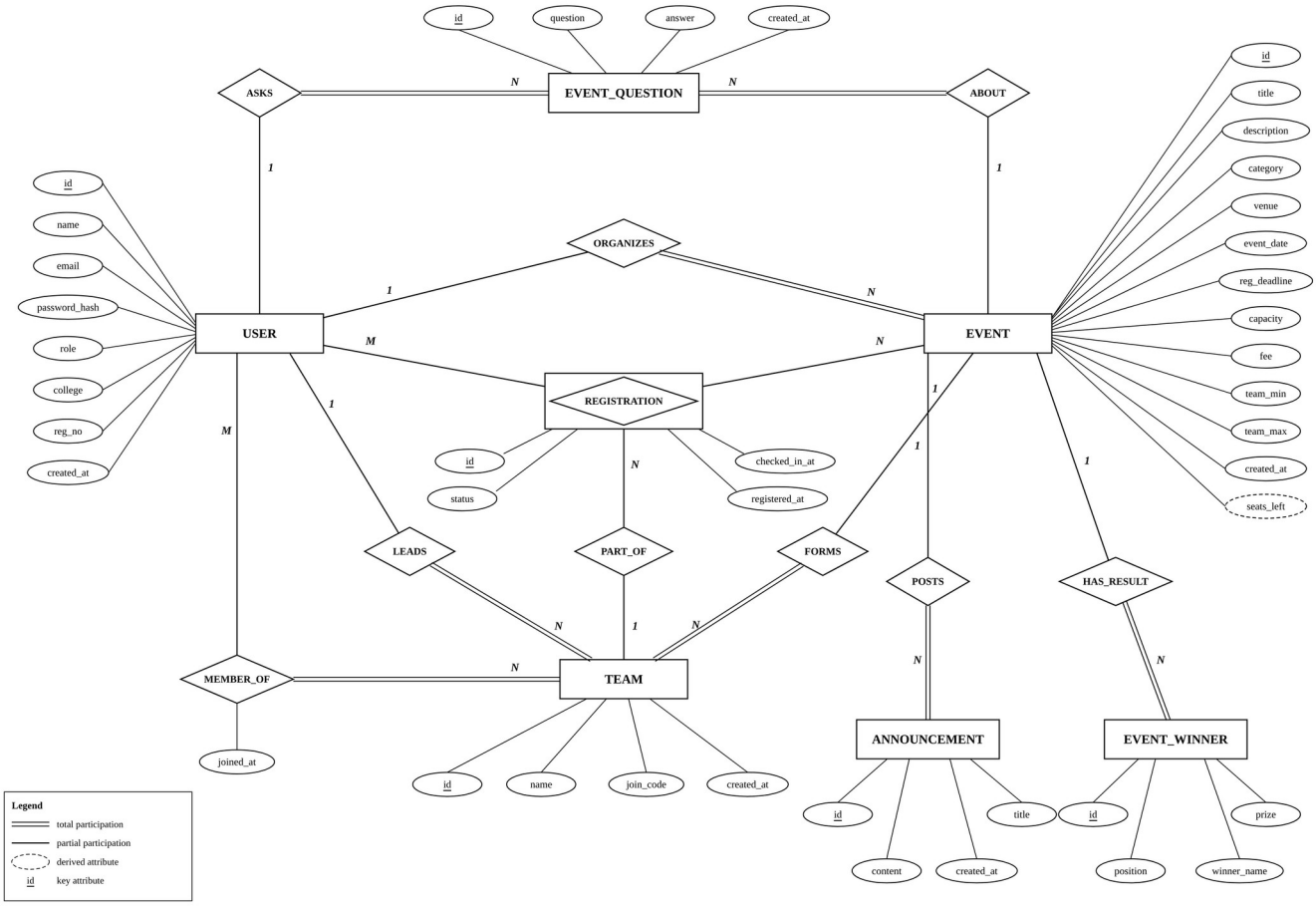


Fig. 2. Entity–Relationship diagram of ShowUp in Chen notation: seven entity sets (REGISTRATION is associative), ten relationship sets, cardinality ratios and participation constraints. The dashed ellipse marks the derived attribute seats_left = capacity − COUNT(confirmed registrations).

D. EER Specialization

Students and organizers share most attributes (name, e-mail, password, college) but take part in different relationships: only students register, join teams and ask questions, and only organizers create events. This is modelled in Fig. 3 as a specialization of the superclass USER into subclasses STUDENT and ORGANIZER on the defining attribute *role*. The specialization is **disjoint** (a user cannot be both) and **total** (every user is one of the two). STUDENT has the specific attribute *reg_no*.

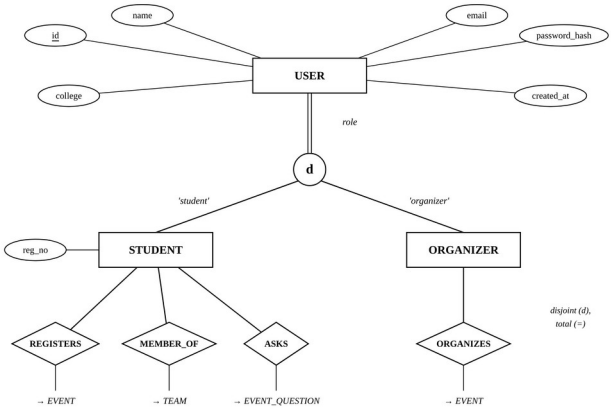


Fig. 3. EER specialization of USER into STUDENT and ORGANIZER (disjoint, total, attribute-defined on role) and the relationships in which each subclass participates.

E. Design Assumptions

- Capacity counts *people*, not teams, so every team member holds an individual registration and ticket.
- A join code identifies a team across the whole site, so it is unique globally; a team name need only be unique within its event.
- A cancelled registration is retained (status = cancelled) as history; re-registering reactivates the same row.
- Winners are stored by name (a team or a person) because results may include external teams.

IV. ER Diagram to Relational Model

The standard seven-step ER-to-relational mapping algorithm [1] plus Step 8 for specialization was applied. Surrogate integer keys (`INT AUTO_INCREMENT`) were chosen for every strong entity because natural keys such as e-mail may change and would make foreign keys wide.

TABLE VI
APPLICATION OF THE MAPPING ALGORITHM

Step	Rule	Result in ShowUp
1	Strong entity → relation	users, events, teams, announcements, event_questions, event_winners
2	Weak entity → relation with owner key	none (all entities have surrogate keys)
3	1:1 relationship	none present
4	1:N → FK on N side	events.organizer_id, teams.event_id, teams.leader_id, announcements.event_id, event_questions.event_id / user_id, event_winners.event_id, registrations.team_id
5	M:N → new relation	team_members(team_id, user_id) with composite PK; registrations (associative entity) with UNIQUE(user_id, event_id)
6	Multivalued attribute	none; repeating groups removed (Section V)
7	n-ary relationship	none
8	Specialization	Option 8C: single relation users with type attribute role ENUM('student','organizer'); reg_no nullable

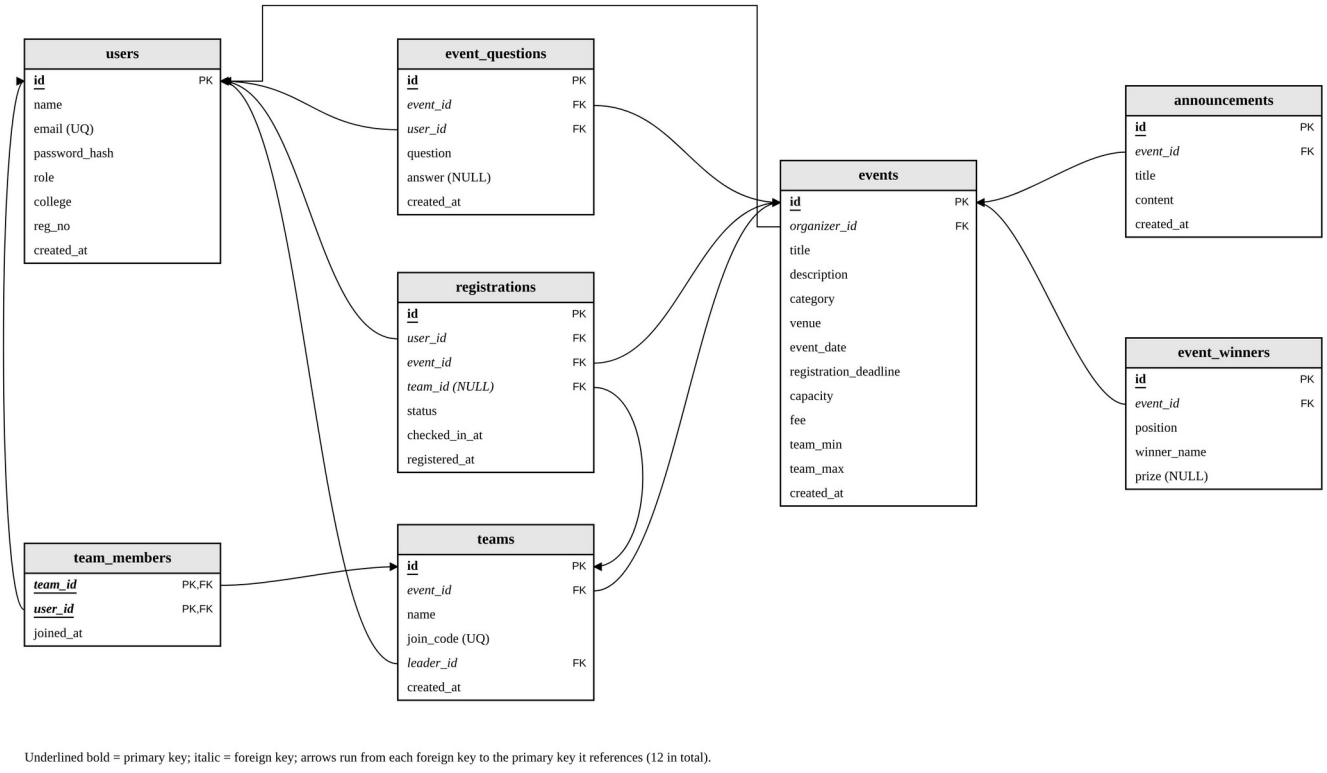
Option 8C (single relation with a type attribute) was chosen over separate subclass tables because the subclasses have only one specific attribute, the specialization is disjoint, and authentication must look a user up by e-mail regardless of role. The cost is that *reg_no* is NULL for organizers and that role-specific participation (e.g. only students register) is enforced in the application tier.

A. Resulting Relational Schema

Primary keys are underlined and foreign keys are shown in italics and marked with †:

users(id, name, email, password_hash, role, college, reg_no, created_at)
events(id, *organizer_id*†, title, description, category, venue, event_date, registration_deadline, capacity, fee, team_min, team_max, created_at)
teams(id, *event_id*†, name, join_code, *leader_id*†, created_at)
team_members(*team_id*†, *user_id*†, joined_at)
registrations(id, *user_id*†, *event_id*†, *team_id*†, status, checked_in_at, registered_at)
announcements(id, *event_id*†, title, content, created_at)
event_questions(id, *event_id*†, *user_id*†, question, answer, created_at)
event_winners(id, *event_id*†, position, winner_name, prize)

Fig. 4 shows the schema diagram with every referential integrity constraint, and Table VII lists the twelve foreign keys with their delete rules.



Underlined bold = primary key; italic = foreign key; arrows run from each foreign key to the primary key it references (12 in total).

Fig. 4. Relational schema diagram. Each arrow is a FOREIGN KEY from the referencing column to the referenced primary key.

TABLE VII
REFERENTIAL INTEGRITY CONSTRAINTS

Foreign key	References	On delete
events.organizer_id	users.id	CASCADE
teams.event_id	events.id	CASCADE
teams.leader_id	users.id	CASCADE
team_members.team_id	teams.id	CASCADE
team_members.user_id	users.id	CASCADE
registrations.user_id	users.id	CASCADE
registrations.event_id	events.id	CASCADE
registrations.team_id	teams.id	SET NULL

Foreign key	References	On delete
announcements.event_id	events.id	CASCADE
event_questions.event_id	events.id	CASCADE
event_questions.user_id	users.id	CASCADE
event_winners.event_id	events.id	CASCADE

CASCADE expresses existence dependency: deleting an event deletes its teams, memberships, registrations, announcements, questions and winners in a single statement. `registrations.team_id` uses SET NULL because a registration may legitimately outlive its team (it then becomes a solo registration rather than disappearing).

V. Normalization (1NF, 2NF, 3NF)

Normalization was carried out in two directions. First, the *bottom-up* view: the flat sign-up sheet used by clubs before ShowUp was normalized step by step, which independently produces the same relations as the ER mapping. Second, the *verification* view: the functional dependencies (FDs) of every final relation were listed and checked against the definitions of 2NF, 3NF and BCNF [1], [5].

A. Unnormalized Form (UNF)

A typical club spreadsheet has one row per team with the members typed into one cell:

```
EventSignupSheet(event_title, event_date, venue,
  organizer_name, organizer_college, team_name,
  join_code, {member_name, email, reg_no,
  member_college, status, checked_in})

CodeSprint 24 | … | NeuralCraft | NC8821 |
  {Ananya, ananya@…, 25BCE0101, VIT Vellore, …;
   Rohan,  rohan@…,  25BCE0102, VIT Vellore, …}
```

The braces mark a *repeating group*, so the relation is not even in 1NF; announcements, questions and winners would form further repeating groups per event.

B. First Normal Form

1NF requires atomic attribute values and no repeating groups. The member group is flattened into one row per (event, member), a surrogate *event_id* is introduced, and the primary key becomes (event_id, member_email). Per-event repeating groups (announcements, questions, winners) are moved to their own relations keyed by their own id with *event_id* as a foreign key.

```
Signup_1NF(event_id, member_email, event_title,
  event_date, venue, organizer_email,
  organizer_name, organizer_college, team_name,
  join_code, member_name, reg_no,
  member_college, status, checked_in)
```

C. Second Normal Form

2NF forbids partial dependencies of non-prime attributes on part of a composite key. In Signup_1NF:

```
FD1: event_id -> event_title, event_date, venue,
      organizer_email, organizer_name,
      organizer_college          (partial)
FD2: member_email -> member_name, reg_no,
      member_college             (partial)
FD3: event_id, member_email -> team_name,
      join_code, status, checked_in   (full)
```

FD1 and FD2 are removed by decomposition into EVENT(event_id, …), MEMBER(member_email, …) and SIGNUP(event_id, member_email, team_name, join_code, status, checked_in). The decomposition is lossless because each common attribute is a key of one of the projections.

D. Third Normal Form

3NF forbids transitive dependencies of non-prime attributes on the key. Two remain:

```
FD4: event_id -> organizer_email
     organizer_email -> organizer_name,
                        organizer_college
FD5: event_id, member_email -> team_name
     event_id, team_name -> join_code, leader
```

FD4 is removed by observing that organizers are themselves users: EVENT keeps only `organizer_id` (a foreign key to USER), and the organizer's name and college are obtained with a JOIN. This is exactly the change made during the project's simplification phase, when event-level copies of the host college, state and city were dropped. FD5 is removed by creating TEAM(id, event_id, name, join_code, leader_id) and TEAM_MEMBERS(team_id, user_id); SIGNUP becomes REGISTRATIONS(id, user_id, event_id, team_id, status, checked_in_at, registered_at). MEMBER and organizer data merge into USERS with the attribute *role*, matching the EER specialization.

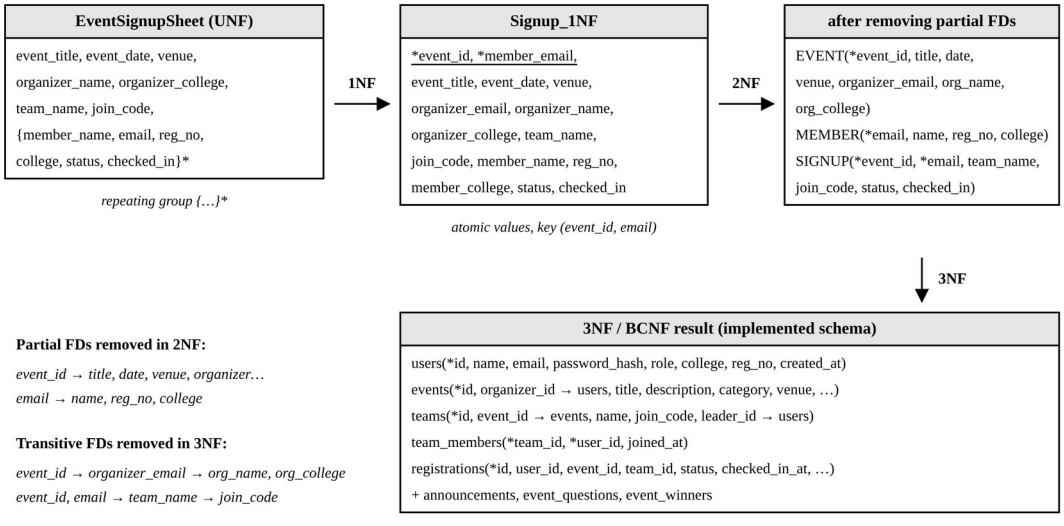


Fig. 5. Normalization of the club sign-up sheet from UNF through 1NF and 2NF to the 3NF/BCNF schema that is implemented.

E. Verification of the Final Schema (BCNF)

Table VIII lists the non-trivial FDs that hold in each implemented relation. In every case the determinant of each FD is a candidate key; hence every relation is in BCNF and therefore also in 3NF and 2NF. All attributes are atomic (1NF).

TABLE VIII
FUNCTIONAL DEPENDENCIES AND NORMAL FORM

Relation	Candidate keys and FDs	NF
users	id → all; email → all (email UNIQUE)	BCNF
events	id → all	BCNF
teams	id → all; join_code → all; (event_id, name) → all	BCNF
team_members	(team_id, user_id) → joined_at	BCNF
registrations	id → all; (user_id, event_id) → all	BCNF
announcements	id → all	BCNF
event_questions	id → all	BCNF
event_winners	id → all	BCNF

F. Derived Data and Controlled Redundancy

Seats left is *not* stored: it is computed as `capacity − COUNT(*)` of confirmed registrations. Storing it would create an update anomaly whenever a registration is inserted, cancelled or reactivated. One controlled redundancy remains across relations: for a team registration, the pair (user, team) appears both in `team_members` and as `registrations.team_id`. It is not a normal-form violation (each relation is BCNF) but an inter-relational dependency; it is kept so that capacity and tickets are per person while team size is counted from `team_members`. Both rows are always written or removed inside the same transaction (Section IX), so they cannot diverge.

VI. DATABASE DESIGN AND TABLE CREATION (DDL)

The schema is created by the script `sql/schema.sql`. Tables are created parent-first (users → events → teams → team_members → registrations → announcements → event_questions → event_winners) so that every foreign key references an existing table. Every statement uses `CREATE TABLE IF NOT EXISTS`, so the script is idempotent and never destroys data; a separate `reset.sql` drops the tables for local development only. Table IX is the data dictionary and Listing 1 the DDL.

TABLE IX
DATA DICTIONARY

Table	Column(s)	Data type	Constraints	Description
users	id	INT	PK, AUTO_INCREMENT	Surrogate key
	name	VARCHAR(100)	NOT NULL	Full name / club name
	email	VARCHAR(255)	NOT NULL, UNIQUE	Login identifier
	password_hash	VARCHAR(255)	NOT NULL	bcrypt hash, never the password
	role	ENUM	NOT NULL, DEFAULT 'student'	'student' | 'organizer' (specialization)
	college / reg_no	VARCHAR(150) / (30)	NULL	Optional profile data
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation
events	id	INT	PK, AUTO_INCREMENT	Surrogate key
	organizer_id	INT	NOT NULL, FK → users	Owning organizer
	title / description	VARCHAR(200) / TEXT	NOT NULL	Content shown to students
	category / venue	VARCHAR(50) / (200)	NOT NULL	Hackathon, Workshop, …
	event_date	DATETIME	NOT NULL	Start (UTC)
	registration_deadline	DATETIME	NOT NULL	Registrations close
	capacity	INT	NOT NULL, CHECK > 0	Max confirmed people
	fee	INT	DEFAULT 0, CHECK ≥ 0	Informational (₹)
	team_min / team_max	INT	DEFAULT 1, CHECK 1 ≤ min ≤ max	team_max = 1 ⇒ solo event
teams	id	INT	PK, AUTO_INCREMENT	Surrogate key
	event_id	INT	NOT NULL, FK → events	Event entered
	name	VARCHAR(100)	NOT NULL, UNIQUE(event_id, name)	Team name
	join_code	VARCHAR(10)	NOT NULL, UNIQUE	6-char code shared with friends
	leader_id	INT	NOT NULL, FK → users	Current leader
team_members	team_id, user_id	INT, INT	composite PK; FKs	M:N users ↔ teams
	joined_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Used for leader succession
registrations	id	INT	PK, AUTO_INCREMENT	Ticket number in QR code

Table	Column(s)	Data type	Constraints	Description
	user_id / event_id	INT	NOT NULL, FKs; UNIQUE(user_id, event_id)	One row per person per event
	team_id	INT	NULL, FK → teams (SET NULL)	NULL for solo
	status	ENUM	DEFAULT 'confirmed'	'confirmed' | 'cancelled'
	checked_in_at	DATETIME	NULL	Attendance timestamp
announcements	id, event_id, title, content, created_at	INT, INT, VARCHAR(150), TEXT, TIMESTAMP	PK; FK → events; NOT NULL	Organizer notices
event_questions	id, event_id, user_id, question, answer	INT, INT, INT, TEXT, TEXT	PK; FKs; answer NULL	Student Q&A
event_winners	id, event_id, position, winner_name, prize	INT, INT, INT, VARCHAR(150), VARCHAR(100)	PK; FK; CHECK position ≥ 1	Results podium

Listing 1. DDL for the ShowUp database (sql/schema.sql, comments omitted).

```
CREATE DATABASE IF NOT EXISTS showup
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
USE showup;

CREATE TABLE IF NOT EXISTS users (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  email         VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role          ENUM('student','organizer')
                NOT NULL DEFAULT 'student',
  college       VARCHAR(150) NULL,
  reg_no        VARCHAR(30)  NULL,
  created_at    TIMESTAMP NOT NULL
                DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS events (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  organizer_id INT          NOT NULL,
  title        VARCHAR(200) NOT NULL,
  description  TEXT         NOT NULL,
  category     VARCHAR(50)  NOT NULL,
  venue        VARCHAR(200) NOT NULL,
  event_date   DATETIME     NOT NULL,
  registration_deadline DATETIME NOT NULL,
  capacity     INT NOT NULL,
  fee          INT NOT NULL DEFAULT 0,
  team_min     INT NOT NULL DEFAULT 1,
  team_max     INT NOT NULL DEFAULT 1,
  created_at   TIMESTAMP NOT NULL
               DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (organizer_id)
    REFERENCES users (id) ON DELETE CASCADE,
  CHECK (capacity > 0),
  CHECK (fee >= 0),
  CHECK (team_min >= 1 AND team_max >= team_min)
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS teams (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  event_id   INT          NOT NULL,
  name       VARCHAR(100) NOT NULL,
  join_code  VARCHAR(10)  NOT NULL UNIQUE,
  leader_id  INT          NOT NULL,
  created_at TIMESTAMP NOT NULL
             DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (event_id, name),
  FOREIGN KEY (event_id)
    REFERENCES events (id) ON DELETE CASCADE,
  FOREIGN KEY (leader_id)
    REFERENCES users (id)  ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS team_members (
  team_id   INT NOT NULL,
  user_id   INT NOT NULL,
  joined_at TIMESTAMP NOT NULL
            DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (team_id, user_id),
  FOREIGN KEY (team_id)
    REFERENCES teams (id) ON DELETE CASCADE,
  FOREIGN KEY (user_id)
    REFERENCES users (id) ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS registrations (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  user_id       INT NOT NULL,
  event_id      INT NOT NULL,
  team_id       INT NULL,
  status        ENUM('confirmed','cancelled')
                NOT NULL DEFAULT 'confirmed',
  checked_in_at DATETIME  NULL,
  registered_at TIMESTAMP NOT NULL
                DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, event_id),
  FOREIGN KEY (user_id)
    REFERENCES users (id)  ON DELETE CASCADE,
  FOREIGN KEY (event_id)
    REFERENCES events (id) ON DELETE CASCADE,
  FOREIGN KEY (team_id)
    REFERENCES teams (id)  ON DELETE SET NULL
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS announcements (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  event_id   INT          NOT NULL,
  title      VARCHAR(150) NOT NULL,
  content    TEXT         NOT NULL,
  created_at TIMESTAMP NOT NULL
             DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (event_id)
    REFERENCES events (id) ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS event_questions (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  event_id   INT  NOT NULL,
  user_id    INT  NOT NULL,
  question   TEXT NOT NULL,
  answer     TEXT NULL,
  created_at TIMESTAMP NOT NULL
             DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (event_id)
    REFERENCES events (id) ON DELETE CASCADE,
  FOREIGN KEY (user_id)
    REFERENCES users (id)  ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS event_winners (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  event_id    INT          NOT NULL,
  position    INT          NOT NULL,
  winner_name VARCHAR(150) NOT NULL,
  prize       VARCHAR(100) NULL,
  FOREIGN KEY (event_id)
    REFERENCES events (id) ON DELETE CASCADE,
  CHECK (position >= 1)
) ENGINE = InnoDB;
```

TABLE X
SUMMARY OF DECLARATIVE CONSTRAINTS

Constraint	Count	Examples
PRIMARY KEY	8	7 surrogate INT keys; composite (team_id, user_id)
FOREIGN KEY	12	11 CASCADE, 1 SET NULL (Table VII)
UNIQUE	4	users.email, teams.join_code, (event_id, name), (user_id, event_id)
CHECK	4	capacity > 0, fee ≥ 0, 1 ≤ team_min ≤ team_max, position ≥ 1
ENUM domain	2	users.role, registrations.status
DEFAULT	11	role, status, fee, team sizes, all timestamps

InnoDB was selected because it supports foreign keys, transactions and row-level locking. MySQL 8.0.16 or later is required because earlier versions parse but silently ignore CHECK constraints [7]. InnoDB automatically creates an index for every primary key, UNIQUE constraint and foreign key, so all join columns used by the application are indexed without additional DDL. The created structure was verified with `SHOW TABLES`, `DESCRIBE` and a query on `information_schema.REFERENTIAL_CONSTRAINTS`, which returned the twelve foreign keys with the expected delete rules.

VII. DATA POPULATION (DML)

Sample data is loaded by `sql/seed.sql`. Ids are written explicitly so that foreign keys in later INSERT statements are easy to follow, and event dates are expressed relative to `CURDATE()` so the demonstration always contains upcoming, open events as well as one completed event with results. Every demo account uses the password *password123*; only its bcrypt hash is stored. Table XI summarizes the data set.

TABLE XI
SAMPLE DATA LOADED BY SEED.SQL

Table	Rows	Content
users	8	3 organizers (VIT Vellore, VIT Chennai, IIT Madras), 5 students
events	7	2 hackathons, workshop, 2 competitions, cultural, talk; 1 in the past
teams	3	NeuralCraft (NC8821), BotBusters, ByteForge
team_members	6	Members of the three teams
registrations	9	8 confirmed (3 checked in), 1 cancelled
announcements	3	Mentor schedule, desk timing, results
event_questions	3	2 answered, 1 unanswered (answer IS NULL)
event_winners	4	1st–3rd place and a special mention

Listing 2. Representative INSERT statements from seed.sql (abridged).

```
SET time_zone = '+00:00';
INSERT INTO users
  (id, name, email, password_hash, role,
   college, reg_no) VALUES
  (1, 'Code Cell', 'codecell@showup.local',
   '$2b$10$A8P…', 'organizer', 'VIT Vellore', NULL),
  (4, 'Ananya Sharma', 'ananya@showup.local',
   '$2b$10$A8P…', 'student', 'VIT Vellore',
   '25BCE0101');

INSERT INTO events (id, organizer_id, title,
  description, category, venue, event_date,
  registration_deadline, capacity, fee,
  team_min, team_max) VALUES
  (1, 1, 'CodeSprint 24', 'Form a team of 2-4…',
   'Hackathon', 'Technology Tower',
   TIMESTAMP(CURDATE() + INTERVAL 6 DAY,'09:00'),
   TIMESTAMP(CURDATE() + INTERVAL 5 DAY,'23:59'),
   40, 0, 2, 4);

INSERT INTO teams (id, event_id, name,
  join_code, leader_id)
  VALUES (1, 1, 'NeuralCraft', 'NC8821', 4);
INSERT INTO team_members (team_id, user_id)
  VALUES (1, 4), (1, 5);
INSERT INTO registrations (id, user_id,
  event_id, team_id, status) VALUES
  (1, 4, 1, 1, 'confirmed'),
  (2, 5, 1, 1, 'confirmed');
```

A. DML Issued by the Application

At run time the application performs all four kinds of data manipulation. Representative statements (with `?` placeholders bound by the driver) are:

Listing 3. Run-time UPDATE and DELETE statements.

```
-- cancel: keep the row as history
UPDATE registrations
   SET status = 'cancelled', team_id = NULL
 WHERE id = ?;

-- check-in: idempotent, guarded by WHERE
UPDATE registrations
   SET checked_in_at = CURRENT_TIMESTAMP
 WHERE id = ? AND status = 'confirmed'
   AND checked_in_at IS NULL;

-- leader leaves: earliest member succeeds
UPDATE teams SET leader_id =
  (SELECT user_id FROM team_members
    WHERE team_id = ?
    ORDER BY joined_at, user_id LIMIT 1)
 WHERE id = ? AND leader_id = ?;

-- delete event: cascades to 6 child tables
DELETE FROM events WHERE id = ?;
```

B. Retrieval Queries and Results

The queries below exercise inner and outer joins, aggregation, grouping, HAVING and NULL handling. Their output was captured from the seeded MySQL 8.0.46 database and is reproduced in Listings 4–6.

Listing 4. Seats taken per event — INNER JOIN, LEFT JOIN, COUNT, GROUP BY (the query behind GET /api/events).

```
SELECT e.id, e.title, u.name AS organizer, e.capacity, COUNT(r.id) AS registered
  FROM events e
  JOIN users u ON u.id = e.organizer_id
  LEFT JOIN registrations r ON r.event_id = e.id AND r.status = 'confirmed'
 GROUP BY e.id ORDER BY e.event_date;

+----+---------------------------+-------------------------+----------+------------+
| id | title                     | organizer               | capacity | registered |
+----+---------------------------+-------------------------+----------+------------+
|  7 | Hack Night 2026           | Code Cell               |       30 |          3 |
|  2 | Figma to Front-End        | Code Cell               |       60 |          2 |
|  1 | CodeSprint 24             | Code Cell               |       40 |          2 |
|  5 | Founders' Fireside        | Code Cell               |      120 |          0 |
|  3 | Valorant 5v5 Showdown     | Cultural & Esports Club |       70 |          0 |
|  4 | Battle of the Bands       | Cultural & Esports Club |       48 |          0 |
|  6 | Line Follower Championship | Robotics Club           |       30 |          1 |
+----+---------------------------+-------------------------+----------+------------+
```

Listing 5. Members of a team — M:N resolution through team_members.

```
SELECT t.name AS team, t.join_code,
       u.name AS member,
       (u.id = t.leader_id) AS is_leader
  FROM teams t
  JOIN team_members tm ON tm.team_id = t.id
  JOIN users u ON u.id = tm.user_id
 WHERE t.join_code = 'NC8821';

+-------------+-----------+---------------+-----------+
| team        | join_code | member        | is_leader |
+-------------+-----------+---------------+-----------+
| NeuralCraft | NC8821    | Ananya Sharma |         1 |
| NeuralCraft | NC8821    | Rohan Mehta   |         0 |
+-------------+-----------+---------------+-----------+
```

The LEFT JOIN in Listing 4 is essential: events with no registrations (e.g. *Founders' Fireside*) still appear with a count of 0, whereas an inner join would drop them. The condition `r.status = 'confirmed'` is placed in the ON clause rather than WHERE for the same reason.

Listing 6. Attendance per event — COUNT(*) versus COUNT(column) on a nullable column.

```
SELECT e.title, COUNT(*) AS confirmed,
       COUNT(r.checked_in_at) AS checked_in
  FROM registrations r
  JOIN events e ON e.id = r.event_id
 WHERE r.status = 'confirmed'
 GROUP BY e.id, e.title;

+---------------------------+-----------+------------+
| title                     | confirmed | checked_in |
+---------------------------+-----------+------------+
| CodeSprint 24             |         2 |          0 |
| Figma to Front-End        |         2 |          0 |
| Line Follower Championship |         1 |          0 |
| Hack Night 2026           |         3 |          3 |
+---------------------------+-----------+------------+
```

Listing 6 relies on the fact that `COUNT(column)` ignores NULLs, so the same pass yields both confirmed and checked-in totals. Two further queries are used by the organizer screens: teams below the minimum size (`GROUP BY t.id … HAVING COUNT(tm.user_id) < e.team_min`, which correctly returns an empty set on the seed data because every team meets its minimum) and unanswered questions (`WHERE q.answer IS NULL`, which returns Sameer Khan's question about overnight food for CodeSprint 24).

VIII. Front-End Design

The user interface is a single-page application written in React 19 with TypeScript and bundled by Vite; styling uses Tailwind CSS with a dark theme and a coral accent colour. Fig. 1 shows its position in the three-tier architecture. The browser never connects to MySQL: every screen obtains its data through the typed wrapper `src/lib/api.ts`, which attaches the user's token to each HTTP request.

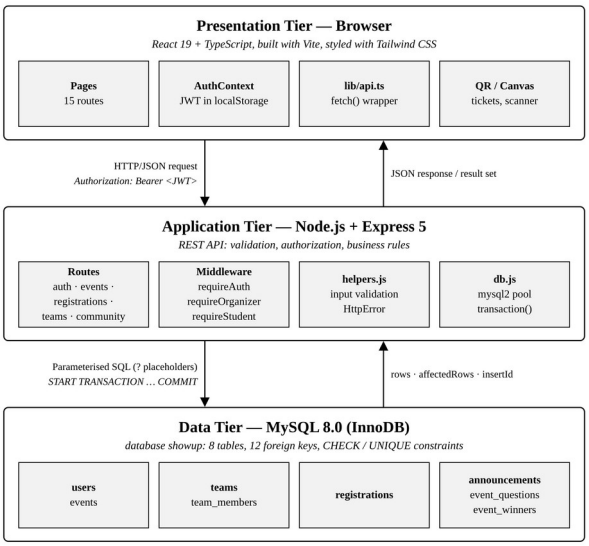


Fig. 1. Three-tier architecture: presentation tier (React), application tier (Express) and data tier (MySQL 8 InnoDB).

A. Design Principles

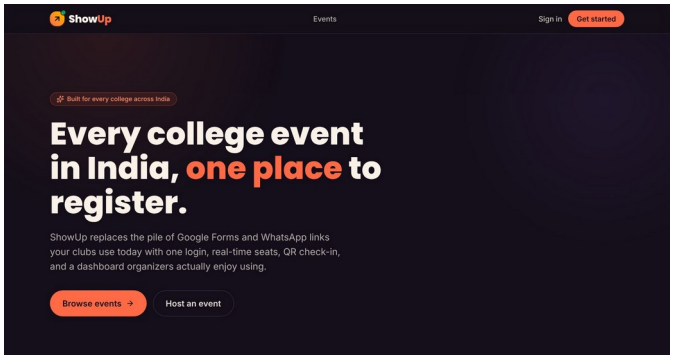
- **Role-aware navigation**—the navbar shows *My Events* to students and *Organizer Studio* to organizers; protected routes redirect signed-out visitors to log in and back.
- **Database values are always live**—seats left, team size and check-in counts are recomputed by SQL on each load rather than cached in the browser.
- **Errors from the server are shown verbatim**—e.g. "Sorry, this event is full."—so constraint violations are understandable to users.
- **Purely client-side conveniences** (bookmarks, calendar export, map distance, PNG pass) are kept out of the database to keep the schema small.

TABLE XII
Pages, Routes and Underlying Data

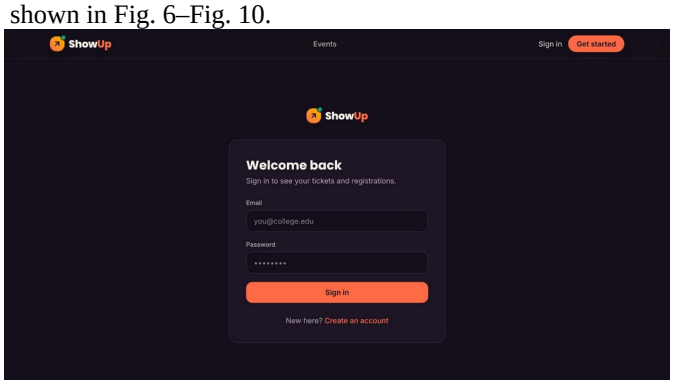
Page (route)	Purpose	Tables read / written
Landing (/)	Introduction, entry points	—
Sign up, Log in	Create account, obtain JWT	users
Events (/events)	Search, filter, sort, bookmark	events ⋈ users ⋈ registrations

Page (route)	Purpose	Tables read / written
Event detail	Overview, Announcements, Q&A, Winners tabs; register / create / join team	events, registrations, teams, team_members, announcements, event_questions, event_winners
My Events (/dashboard)	Tickets, team, cancel, calendar	registrations ⋈ events ⋈ teams
Profile	Account details and counts	users, registrations, team_members
Organizer Studio	Own events with fill rate; edit, delete	events, registrations
Create / Edit event	Validated event form	events
Participants	By team / all, KPIs, CSV export	registrations ⋈ users ⋈ teams
QR Check-in	Camera scan or manual ticket entry	registrations

Screenshots of the running system with the seed data are shown in Fig. 6–Fig. 10.

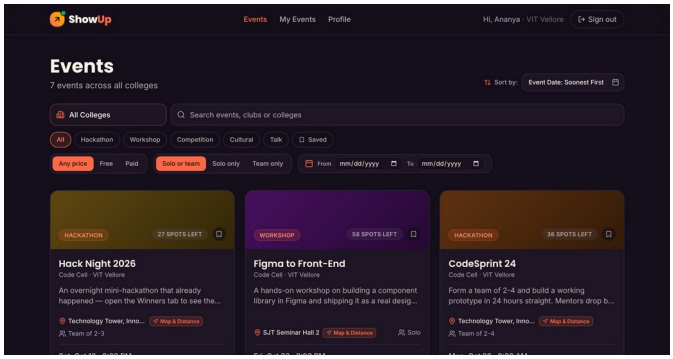


(a) Landing page

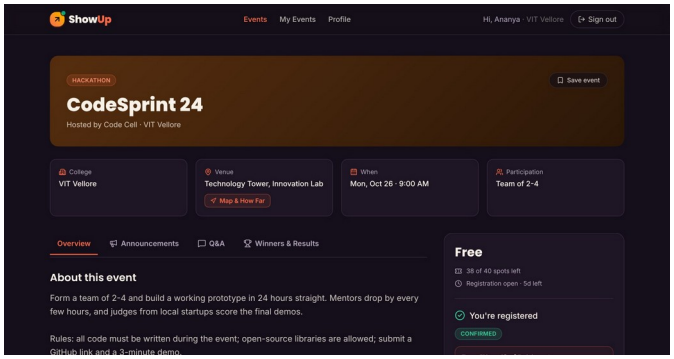


(b) Log-in form

Fig. 6. Public pages. Signing in returns a JWT that the browser stores and sends with every API request.

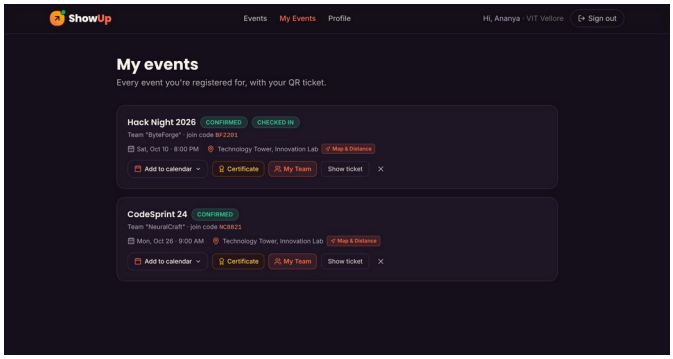


(a) Event directory with filters and live seats left

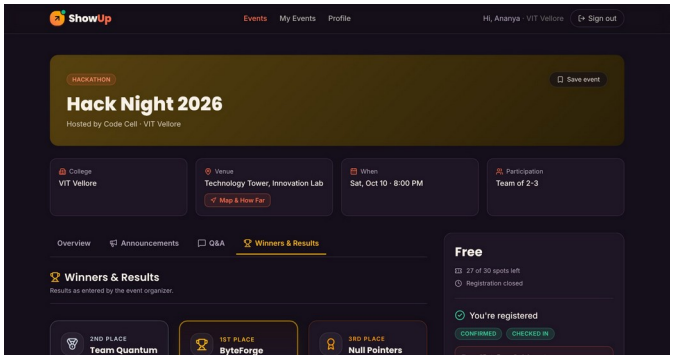


(b) Event detail with team join code

Fig. 7. Student discovery and registration. "27 spots left" on each card is capacity minus the COUNT of confirmed registrations computed by SQL.

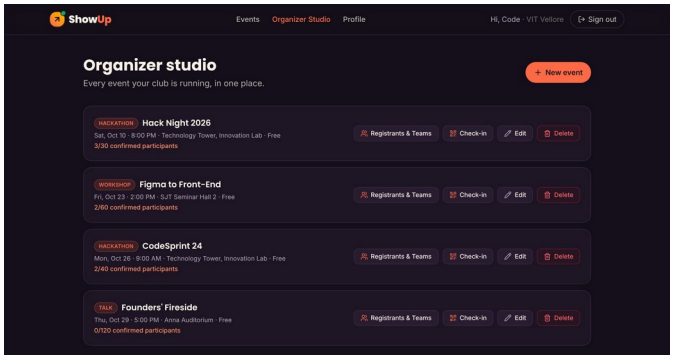


(a) My Events — tickets, team, certificate

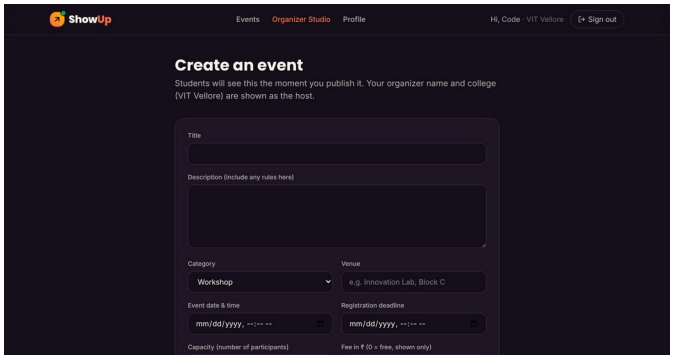


(b) Winners & Results tab

Fig. 8. Student dashboard and results podium read from registrations and event_winners.

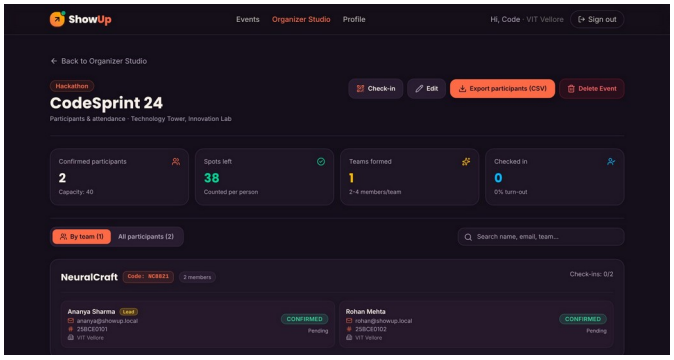


(a) Organizer Studio

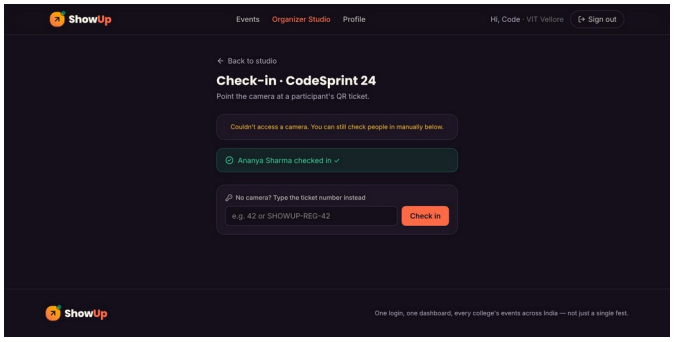


(b) Create-event form

Fig. 9. Organizer pages for event CRUD; the form enforces the same rules as the CHECK constraints.



(a) Participants grouped by team



(b) QR check-in (manual entry shown)

Fig. 10. Attendance management. A successful check-in sets registrations.checked_in_at.

IX. Business Logic Implementation

Business rules are enforced in two layers. Rules that can be expressed declaratively are placed in the schema, so they hold no matter which client modifies the data; rules that depend on time, on aggregates or on the identity of the requester are implemented in the Express route handlers inside transactions. Where possible a rule is enforced in *both* layers—the application gives a friendly message and the constraint is the backstop.

TABLE XIII
Business Rules and Where They Are Enforced

ID	Rule	Enforced by
BR-1	E-mail is unique per account	UNIQUE + API (409)
BR-2	Capacity > 0; fee ≥ 0; 1 ≤ team_min ≤ team_max	CHECK + API (400)
BR-3	Deadline must precede event date	API
BR-4	No registration after the deadline	API (in transaction)
BR-5	One registration per student per event	UNIQUE + API (409)
BR-6	Confirmed registrations ≤ capacity	API: lock + COUNT
BR-7	Solo events: register; team events: create/join	API
BR-8	Team size ≤ team_max when joining	API: COUNT team_members
BR-9	Team names unique within an event	UNIQUE(event_id, name)
BR-10	Only students register; only organizers create events	Middleware
BR-11	Only the owner organizer edits, deletes, checks in	API ownership check
BR-12	Capacity cannot be lowered below current count	API
BR-13	A ticket is checked in at most once	Guarded UPDATE
BR-14	Leader leaves → earliest member leads; empty team deleted	API (in transaction)

A. Registration Transaction

Fig. 11 shows the algorithm for a solo registration. The decisive step is `SELECT … FOR UPDATE` on the event row: InnoDB grants an exclusive row lock that is held until COMMIT or ROLLBACK, so concurrent registrations for the same event are serialized and the capacity check followed by INSERT behaves atomically [14]. Registrations for *different* events lock different rows and proceed in parallel. If any check fails the handler throws, and the transaction helper issues ROLLBACK.

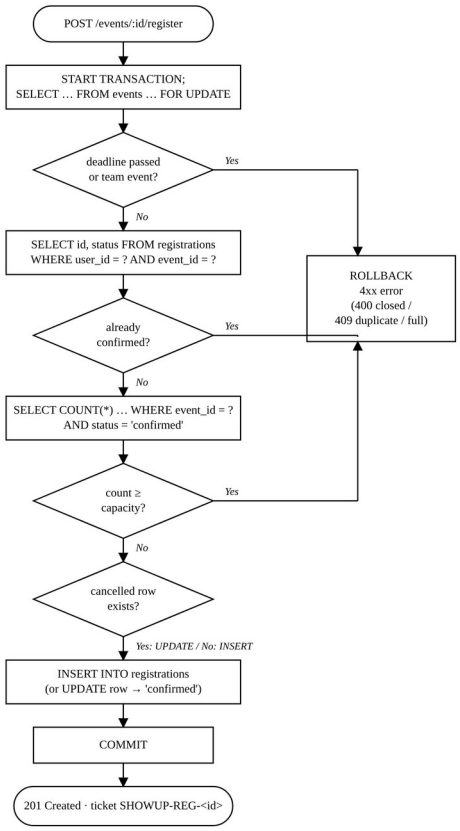


Fig. 11. Flowchart of the solo-registration transaction (POST /api/events/:id/register).

Listing 7. Shared registration steps (server/routes/registrations.js, abridged).

```
async function lockOpenEvent(conn, eventId) {
  const [[event]] = await conn.query(
    "SELECT * FROM events WHERE id = ? FOR UPDATE",
    [eventId]);
  if (!event) throw new HttpError(404, "Event not found.");
  if (new Date(event.registration_deadline) < new Date())
    throw new HttpError(400, "Registration … closed.");
  return event;
}
async function ensureSeatAvailable(conn, event) {
  const [[{ taken }]] = await conn.query(
    `SELECT COUNT(*) AS taken FROM registrations
      WHERE event_id = ? AND status = 'confirmed'`,
    [event.id]);
  if (taken >= event.capacity)
    throw new HttpError(409, "Sorry, this event is full.");
}
```

B. Team Creation and Joining

Creating a team locks the event, checks the deadline, duplicate registration and capacity, then inserts into `teams`, `team_members` and `registrations` in one transaction. The join code is six characters drawn with `crypto.randomInt` from a 32-symbol alphabet that omits look-alike characters (0/O, 1/I), giving $32^6 \approx 1.07 \times 10^9$ codes; uniqueness is checked before insertion and guaranteed by the UNIQUE constraint. Joining looks the team up by code, locks its event, verifies that `COUNT(*)` of `team_members` is below `team_max` and that a seat is free, and inserts the membership and registration.

C. Cancellation and Leader Succession

Cancelling sets `status = 'cancelled'` instead of deleting, so the seat is released (seats count only confirmed rows) while history is retained; re-registering reactivates the same row and the ticket number is preserved. If the person belonged to a team, their `team_members` row is deleted; the team is deleted when it becomes empty, and if the leader left, the member with the earliest `joined_at` becomes leader via the correlated sub-query in Listing 3.

D. Authorization and Validation

TABLE XIV
AUTHORIZATION MATRIX

Action	Visitor	Student	Organizer
Browse events / Q&A / winners	✓	✓	✓
Register, create / join team		✓	
Cancel registration		own	own event
Ask a question		✓	✓
Create event			✓
Edit / delete event, check-in			own event
Answer, announce, publish winners			own event
View participant list / team		own team	own event

Middleware `requireAuth` verifies the token and attaches the user's id and role; `requireStudent` and `requireOrganizer` enforce role rules. Ownership is checked against `events.organizer_id`. Request bodies are validated by small helpers (`requireText`, `requireInt`, `requireDate`) that trim input, bound lengths and ranges, and reject invalid categories or dates with HTTP 400.

X. DATABASE CONNECTIVITY

The application tier connects to MySQL with the *mysql2* driver [13] using its promise API. A single connection pool of ten connections is created at start-up from environment variables (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`) read from a git-ignored `.env` file. Every pooled connection sets `time_zone = '+00:00'` so that all DATETIME values are stored and returned in UTC and converted to local time only in the browser.

Listing 8. Connection pool and transaction helper (server/db.js).

```
export const pool = mysql.createPool({
  host, port, user, password, database,
  connectionLimit: 10,
  timezone: "Z",
});

export async function transaction(work) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const result = await work(conn);
    await conn.commit();
    return result;
  } catch (err) {
    await conn.rollback();
    throw err;
  } finally {
    conn.release();
  }
}
```

All statements in a transaction must run on the *same* connection, which is why `transaction()` checks out one connection and passes it to the callback; single-statement reads use the pool directly. The `finally` clause guarantees that the connection is returned to the pool even when an error occurs.

A. Parameterised Queries and SQL-Injection Prevention

No SQL string in the code base is built by concatenating user input. Values are always passed separately and bound to `?` placeholders by the driver, which is the primary defence recommended by OWASP [12]. Test TC-35 (Section XI) confirms that the classic payload `' OR '1'='1` is treated as a literal e-mail address.

B. Request Life Cycle

Fig. 12 traces a complete request—joining a team—from the browser through the API to the database and back, showing the transaction boundaries and the row lock.

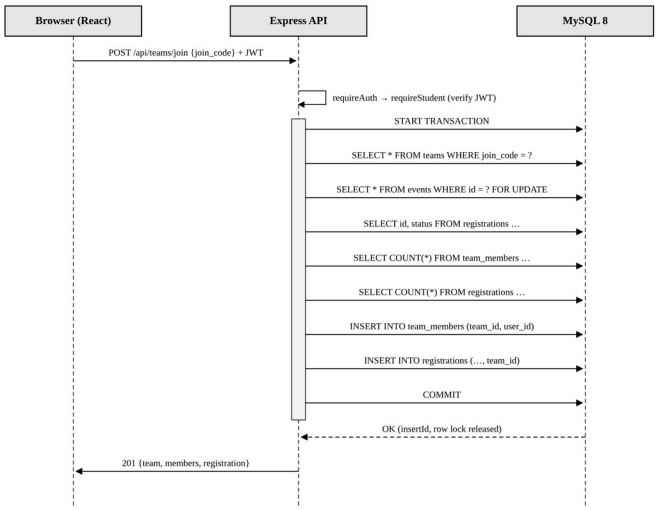


Fig. 12. Sequence diagram for POST /api/teams/join, showing authentication, the transaction and the row-level lock on the event.

C. Authentication

At sign-up the password is hashed with bcrypt (cost factor 10) [11]; at log-in the stored hash is compared and a JSON Web Token [10] containing the user id, name and role is returned, signed with a server secret and valid for seven days. Because the token is self-contained, no sessions table is needed. Logging out simply discards the token in the browser.

D. Error Mapping

A central Express error handler converts errors into JSON responses: application errors keep their HTTP status; MySQL `ER_DUP_ENTRY` (1062) becomes 409 Conflict; connection and missing-table errors become 503 with a hint to run `npm run db:init`; anything else is logged and returned as 500 without exposing internal details.

TABLE XV
REST API ENDPOINTS AND THE SQL THEY EXECUTE

Method and path	Access	SQL operation
POST /api/auth/signup · POST /api/auth/login · GET /api/auth/me	public / public / signed in	INSERT INTO users (bcrypt hash) · SELECT … WHERE email = ? · SELECT … WHERE id = ?
GET /api/events · GET /api/events/:id	public	events ⋈ users ⋈ registrations, COUNT … GROUP BY e.id
POST /api/events · PUT /api/events/:id · DELETE /api/events/:id	organizer (owner for PUT/DELETE)	INSERT INTO events · UPDATE events … WHERE id = ? · DELETE FROM events (cascades)
POST /api/events/:id/register	student	transaction: SELECT … FOR UPDATE, COUNT, INSERT/UPDATE registrations
GET /api/my-registrations	signed in	registrations ⋈ events ⋈ users ⋈ teams
DELETE /api/registrations/:id	participant or owner organizer	transaction: UPDATE status, DELETE team_members, DELETE/UPDATE teams
GET /api/events/:id/registrations	owner organizer	registrations ⋈ users ⋈ teams
POST /api/registrations/:id/check-in	owner organizer	guarded UPDATE registrations SET checked_in_at
POST /api/events/:id/teams · POST /api/teams/join	student	transaction: INSERT teams, team_members, registrations
GET /api/teams/:id	team member or owner organizer	team_members ⋈ users
GET | POST /api/events/:id/announcements	public | owner organizer	SELECT … ORDER BY created_at DESC | INSERT
GET | POST /api/events/:id/questions · PUT /api/questions/:id/answer	public | signed in | owner organizer	event_questions ⋈ users | INSERT | UPDATE … SET answer = ?
GET | POST /api/events/:id/winners	public | owner organizer	SELECT … ORDER BY position | INSERT

XI. Testing and Validation

A. Strategy

Testing followed the structure of IEEE Std 829 [15]: each test case has an identifier, input, expected result, actual result and verdict. Four levels were used:

- **Static checks**—TypeScript compilation (`tsc -b`) and the production build succeed with no errors; the *oxlint* linter reports 0 errors (8 style warnings about React effects).
- **API black-box tests**—a Node.js script sends real HTTP requests to the running server and compares status codes and messages with expectations (TC-01 to TC-27, TC-34, TC-35).
- **Database constraint tests**—invalid statements are executed directly against MySQL to prove that the DBMS itself rejects them, independently of the application (TC-28 to TC-33).
- **Manual UI walkthrough**—every page in Table XII was exercised in Chromium with the seed accounts; the screenshots in Section VIII were captured during this pass.

All tests were run on MySQL Community Server 8.0.46 with Node.js 22 after `npm run db:reset`, so the starting state is exactly the seed data of Table XI.

TABLE XVI
Test Cases and Results

ID	Module	Test description	Input	Expected	Actual	Result
TC-01	Auth	Login with valid credentials	ananya@showup.local / password123	200, JWT returned	200	Pass
TC-02	Auth	Login with wrong password	ananya@… / wrong	401 rejected	401 – "Invalid email or password."	Pass
TC-03	Auth	Sign up new student	new e-mail, 8-char password	201, user created	201	Pass
TC-04	Auth	Sign up with existing e-mail	ananya@showup.local	409 (UNIQUE email)	409 – "An account with this email already exists."	Pass
TC-05	Auth	Sign up with short password	password = 123	400 validation error	400 – "Password must be at least 6 characters."	Pass
TC-06	Auth	Protected route with invalid token	Bearer not-a-token	401 rejected	401 – "Your session has expired. Please sign in again."	Pass
TC-07	Registration	Solo registration for open event	Sameer → Founders' Fireside	201, status confirmed	201	Pass
TC-08	Registration	Duplicate registration	Sameer → same event again	409 already registered	409 – "You're already registered for this event."	Pass
TC-09	Registration	Register after deadline	Priya → Hack Night (past)	400 registration closed	400 – "Registration for this event has closed."	Pass
TC-10	Registration	Solo register on team event	Priya → CodeSprint 24	400 use team flow	400 – "This is a team event — create a team or join one with a code."	Pass
TC-11	Registration	Organizer tries to register	Code Cell → event 5	403 students only	403 – "Only student accounts can register for events."	Pass
TC-12	Registration	Re-register after cancellation	Priya (cancelled row id 9) → event 5	201, same row id 9 reused	201	Pass
TC-13	Teams	Create team with duplicate name	"NeuralCraft" on CodeSprint	409 UNIQUE(event_id,name)	409 – "A team with that name already exists for this event."	Pass
TC-14	Teams	Create team	Priya → "Pixel Pirates"	201, 6-char join code	201	Pass
TC-15	Teams	Join with invalid code	ZZZZZZ	404 no such team	404 – "No team has that join code."	Pass
TC-16	Teams	Join team with code (lower-case)	Sameer → nc8821	201, member added	201	Pass
TC-17	Teams	Join team, 4th member (max 4)	Test Student → NC8821	201, team now full	201	Pass
TC-18	Teams	Join a full team	Vikram → NC8821 (4/4)	409 team full	409 – "This team is full (maximum 4 members)."	Pass
TC-19	Check-in	Check in a confirmed ticket	SHOWUP-REG-10 by Code Cell	200, attendance recorded	200	Pass
TC-20	Check-in	Check in same ticket twice	same ticket	409 already checked in	409 – "Sameer Khan is already checked in."	Pass
TC-21	Check-in	Check in by another organizer	Robotics Club	404 not your event	404 – "That ticket isn't for one of your events."	Pass
TC-22	Events	Deadline after event date	deadline = event + 1 day	400 validation error	400 – "The registration deadline must be before the event starts."	Pass
TC-23	Events	Invalid category	category = Party	400 validation error	400 – "Category must be one of: Hackathon, Workshop, Competition, Cultural, Talk."	Pass
TC-24	Events	Create valid event	capacity = 1	201, event created	201	Pass

ID	Module	Test description	Input	Expected	Actual	Result
TC-25	Events	Edit another organizer's event	Robotics Club → event 1	403 forbidden	403 – "Only the organizer who created this event can change it."	Pass
TC-26	Concurrency	5 simultaneous registrations for 1 seat	Promise.all × 5	exactly 1 × 201, 4 × 409	1 × 201, 4 × 409 – "Sorry, this event is full."	Pass
TC-27	Teams	Team leader cancels registration	Ananya cancels reg 1	status cancelled; leadership passes to Rohan (id 5)	200; leader_id = 5	Pass
TC-28	DB constraint	CHECK capacity > 0	INSERT event with capacity = 0	rejected by CHECK	ER_CHECK_CONSTRAINT_VIOLATED (3819)	Pass
TC-29	DB constraint	CHECK team_max >= team_min	UPDATE events SET team_min = 5, team_max = 2	rejected by CHECK	ER_CHECK_CONSTRAINT_VIOLATED (3819)	Pass
TC-30	DB constraint	FOREIGN KEY on registrations.event_id	INSERT registration with event_id = 999	rejected by FK	ER_NO_REFERENCED_ROW_2 (1452)	Pass
TC-31	DB constraint	UNIQUE (user_id, event_id)	INSERT duplicate (user 5, event 1)	rejected as duplicate	ER_DUP_ENTRY (1062)	Pass
TC-32	DB constraint	ENUM on users.role	INSERT user with role = 'admin'	rejected (strict mode)	WARN_DATA_TRUNCATED (1265)	Pass
TC-33	DB constraint	NOT NULL on events.title	INSERT event with title = NULL	rejected	ER_BAD_NULL_ERROR (1048)	Pass
TC-34	DB constraint	ON DELETE CASCADE from events	DELETE /api/events/7	child rows removed	teams 1→0, regs 3→0, ann. 1→0, winners 4→0	Pass
TC-35	Security	SQL injection in login	email = ' OR '1'='1	401, treated as plain text	401 – "Invalid email or password."	Pass

TABLE XVII
TEST SUMMARY

Module	Cases	Passed
Auth	6	6
Registration	6	6
Teams	7	7
Check-in	3	3
Events	4	4
Concurrency	1	1
DB constraint	7	7
Security	1	1
Total	**35**	**35**

B. Concurrency Validation

TC-26 is the most important test for a registration system. An event with capacity 1 was created and five newly registered students sent their registration requests simultaneously with `Promise.all`. Exactly one request returned 201 Created and the other four returned 409 "Sorry, this event is full." Without the `FOR UPDATE` lock, all five transactions could read a count of 0 before any of them inserted, and the event would be over-subscribed five times—the classic lost-update anomaly.

C. Integrity Validation

TC-28 to TC-33 show that the DBMS rejects invalid data even if the application were bypassed: CHECK violations raise error 3819, a dangling foreign key raises 1452, a duplicate registration raises 1062, a value outside the ENUM domain raises 1265 in strict mode, and a NULL title raises 1048. TC-34 deletes an event through the API and confirms that its 1 team, 3 registrations, 1 announcement and 4 winners were removed by ON DELETE CASCADE. TC-27 confirms leader succession: when Ananya (leader of NeuralCraft) cancelled, `teams.leader_id` changed to Rohan (id 5).

XII. PROJECT DEMONSTRATION AND DOCUMENTATION

A. Installation

Listing 9. Commands to install and run ShowUp locally.

```
npm install
cp .env.example .env   # DB_USER, DB_PASSWORD,
                       # JWT_SECRET
npm run db:reset       # schema.sql + seed.sql
npm run dev            # API :3001, web :5173
```

The same SQL files can be run from MySQL Workbench or the command line (`mysql -u root -p < sql/schema.sql`). The script runner refuses to execute `reset.sql` unless the host is local and `NODE_ENV` is not production, preventing accidental data loss.

TABLE XVIII
DEMONSTRATION ACCOUNTS (PASSWORD: PASSWORD123)

Account	Role and purpose
codecell@showup.local	Organizer, VIT Vellore; owns CodeSprint 24, workshop, Hack Night
culturals@showup.local	Organizer, VIT Chennai
robotics@showup.local	Organizer, IIT Madras
ananya@ / rohan@showup.local	Students in team NeuralCraft (NC8821)
priya@ / vikram@ / sameer@	Students (…@showup.local)

B. Demonstration Script

The live demonstration walks through the full life cycle while showing the effect of each action in MySQL Workbench:

1) Run `SHOW TABLES` and `DESCRIBE registrations` to present the schema.

2) Sign in as *sameer@showup.local*, open CodeSprint 24 and *Join with code* NC8821; show the new rows in `team_members` and `registrations`.
3) Attempt to join again to show the 409 duplicate message and the UNIQUE(user_id, event_id) backstop.
4) Open *My Events* and display the QR ticket (SHOWUP-REG-<id>).
5) Sign in as *codecell@showup.local*, open *Participants* to see the team grouping, then *Check-in* and enter Sameer's ticket; show `checked_in_at` updated.
6) Run the queries of Listings 4–6 to show live aggregates.
7) Delete a test event and show that child rows disappear (ON DELETE CASCADE).

C. Documentation Deliverables

- **README.md**—features, setup, database design, every foreign key, example queries, API table and project structure.
- **sql/schema.sql, seed.sql, reset.sql**—commented DDL and DML scripts.
- **Inline code comments**—each route explains the SQL it runs and why (e.g. why the event row is locked).
- **This report**—requirements, ER/EER, mapping, normalization, implementation and test evidence.
- **Git history**—commits on GitHub record the evolution from an earlier 12-table Supabase design to the present 8-table MySQL design.

D. Repository Structure

```
sql/      schema.sql, seed.sql, reset.sql
server/   index.js, db.js, auth.js, helpers.js,
          run-sql.js
  routes/ auth, events, registrations,
          teams, community
src/      lib/api.ts, types/, context/,
          components/, pages/, pages/organizer/
```

XIII. Conclusion and Future Work

ShowUp demonstrates the complete database development life cycle on a real, everyday problem. Starting from the deficiencies of form-and-spreadsheet registration, the requirements were analysed and modelled as an ER diagram with an EER specialization, mapped to eight relations, and shown to be in BCNF. The resulting MySQL schema enforces integrity declaratively through twelve foreign keys and CHECK, UNIQUE and ENUM constraints, while the application tier enforces time- and aggregate-dependent rules inside ACID transactions with row-level locking. All 35 test cases passed, including a concurrency test that confirms an event cannot be over-subscribed.

Possible extensions, each of which maps to a further DBMS topic, include: a waitlist table with automatic promotion implemented by a trigger or stored procedure; views for organizer reports; a secondary index on `events.event_date` and a FULLTEXT index for search as data grows; an audit table populated by triggers; e-mail notifications through an outbox table; and replication with read replicas for large inter-college festivals.

Acknowledgment

The author thanks the faculty of the Database Management Systems course at Vellore Institute of Technology for their guidance, and the student club coordinators who described how event registration is handled today.

References

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[3] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[4] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976.

[5] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

[6] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[7] Oracle Corporation, "MySQL 8.0 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[8] OpenJS Foundation, "Express 5.x API reference." [Online]. Available: https://expressjs.com/en/5x/api.html

[9] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[11] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[12] OWASP Foundation, "SQL Injection Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[13] A. Sidorov *et al.*, "node-mysql2: Fast MySQL driver for Node.js." [Online]. Available: https://github.com/sidorares/node-mysql2

[14] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.

[15] *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008, 2008.